AI Платформа AIналитик

Скачать: https://github.com/chaussky/ainalyst.git

Руководство разработчика

Версия: v1 / Дата: апрель 2026

Этот документ описывает архитектуру платформы AIналитик «под капотом»: как устроены компоненты, как они взаимодействуют, как добавить новый сервер или скилл. Для понимания того, **как пользоваться** платформой — смотрите пользовательскую инструкцию (`1-introduction.md` и `ch*` файлы).

Содержание

1. Архитектура платформы
2. Система фаз и `phase.py`
3. `common.py` — единый источник истины
4. Архитектура MCP-серверов
5. Структура скиллов (SKILL.md)
6. Хранилище артефактов
7. Интеграция с Confluence
8. Тестирование
9. Среда разработки
10. Добавление нового MCP-сервера
11. Технический долг и ADR-реестр

---

1. Архитектура платформы

Три слоя

Платформа состоит из трёх слоёв, которые работают вместе при каждом запросе BA:

```
┌──────────────────────────────────────────────────────┐
│  Claude Code                                         │
│  Агент. Читает CLAUDE.md и SKILL.md, управляет       │
│  фазами, вызывает MCP-инструменты, ведёт BA          │
│  по процессу.                                        │
└──────────────────────────┬───────────────────────────┘
                           │ вызывает
┌──────────────────────────▼───────────────────────────┐
│  Skills (SKILL.md + references/)                     │
│  21 специализированный модуль знаний. Каждый         │
│  «знает» одну задачу BABOK: методологию, алгоритм,   │
│  шаблоны, ссылки на MCP-инструменты.                 │
└──────────────────────────┬───────────────────────────┘
                           │ инструктирует вызывать
┌──────────────────────────▼───────────────────────────┐
│  MCP-серверы (22 × *_mcp.py)                         │
│  111 инструментов. Выполняют аналитические           │
│  операции: строят графы требований, анализируют      │
│  транскрипты, сохраняют артефакты.                   │
└──────────────────────────┬───────────────────────────┘
                           │ пишет
┌──────────────────────────▼───────────────────────────┐
│  governance_plans/data/    — JSON (машиночитаемые)   │
│  governance_plans/reports/ — Markdown (для людей)    │
└──────────────────────────────────────────────────────┘
```

Поток одного запроса

Когда BA пишет что-то вроде «подготовь план выявления для интервью с финдиром», происходит следующее:

1. **Claude Code** читает `CLAUDE.md` — системный промпт, описывающий роль и принципы работы
2. По триггерам в `CLAUDE.md` определяет задачу: это **4.1 Подготовка к выявлению**
3. Читает `skills/elicitation_prep/SKILL.md` — методологию и алгоритм
4. Задаёт уточняющие вопросы (тип встречи, стейкхолдер, цель)
5. Вызывает MCP-инструмент `save_elicitation_plan` из `elicitation_mcp.py`
6. Инструмент через `common.py` сохраняет артефакт в `governance_plans/reports/`
7. Claude Code сообщает BA о результате и предлагает следующий шаг

Ключевые файлы

Файл / папка	Роль
`CLAUDE.md`	Системный промпт для Claude Code: роль, фазы, триггеры по задачам, принципы
`phase.py`	Генератор `.mcp.json`. Управляет активной фазой.
`skills/common.py`	Единый источник истины: константы, матрицы, `save_artifact`, пути к папкам
`skills/*_mcp.py`	22 MCP-сервера, по одному на задачу BABOK
`skills/integrations/confluence_mcp.py`	BASE_SERVER — Confluence, включён во все фазы
`skills/*/SKILL.md`	Контекстные инструкции для Claude Code (методология + алгоритм)
`skills/*/references/*.md`	Детальные справочники — читаются только по алгоритму SKILL.md
`governance_plans/data/`	JSON-артефакты: граф требований, результаты приоритизации и т.д.
`governance_plans/reports/`	Markdown-артефакты: планы, протоколы, рекомендации
`inputs/`	Входные материалы BA: транскрипты, документы, регламенты
`.claude/`	Хуки, правила и настройки Claude Code
`.mcp.json`	Генерируется `phase.py`. В `.gitignore` — пути специфичны для машины

---

2. Система фаз и `phase.py`

Проблема и решение

У LLM есть ограниченное контекстное окно. Загрузка всех 22 MCP-серверов одновременно занимает значительную его часть и деградирует качество работы. Платформа решает это через **активную фазу**: в каждый момент загружены только серверы нужной главы BABOK.

`phase.py` управляет этим: читает нужный набор серверов → генерирует `.mcp.json` с абсолютными путями → записывает имя активной фазы в `.ainalyst_phase`.

Фазы

Имя фазы	Команда	Главы BABOK	MCP-серверы (кроме BASE)
`planning`	`python phase.py planning`	3	— только BASE
`elicitation`	`python phase.py elicitation`	4.1–4.5	5 серверов
`lifecycle`	`python phase.py lifecycle`	5.1–5.5	5 серверов
`analysis`	`python phase.py analysis`	6.1–6.4	4 сервера
`design`	`python phase.py design`	7.1–7.6	6 серверов
`full`	`python phase.py full`	Все главы	20 серверов

Без аргументов `python phase.py` показывает текущую активную фазу и подсказки по каждой.

BASE_SERVER

Два сервера присутствуют во **всех** фазах:

```
BASE_SERVER = {
      "babok-ch3": _server("skills/planning_mcp.py"),
      "babok-confluence": _server("skills/integrations/confluence_mcp.py"),
  }
```

- `planning_mcp.py` — лёгкий (6 инструментов), нужен всегда: `project_id`, реестр стейкхолдеров
- `confluence_mcp.py` — 4 инструмента, стартует без `.env`. Ошибка возникает только при вызове инструмента, если ключи не заполнены

Как `phase.py` генерирует пути

```
PROJECT_ROOT = Path(__file__).resolve().parent


  def _server(script: str) -> dict:
      return {"command": "python", "args": [str(PROJECT_ROOT / script)]}
```

`Path(__file__).resolve().parent` — абсолютный путь к папке где лежит `phase.py`. Это делает `.mcp.json` переносимым на уровне скрипта: путь вычисляется в момент запуска `phase.py` на конкретной машине. Поэтому `.mcp.json` в `.gitignore` — файл специфичен для каждой машины разработчика.

Жизненный цикл смены фазы

```
python phase.py design     # 1. Записывает .ainalyst_phase, генерирует .mcp.json
  # Claude Code: /restart    # 2. Перезагружает MCP-серверы из нового .mcp.json
```

После `/restart` Claude Code загружает новый набор серверов — и работает с инструментами нужной главы.

Чтение текущей фазы

`phase.py` без аргументов читает `.ainalyst_phase` и выводит что сейчас активно. Файл `.ainalyst_phase` тоже в `.gitignore` — он специфичен для текущей сессии разработчика.

---

3. `common.py` — единый источник истины

Файл `skills/common.py` — центральный служебный модуль. Все MCP-серверы импортируют из него константы и функции. Это гарантирует что бизнес-логика не дублируется.

Пути к артефактам

```
BASE_DIR    = "governance_plans"
  DATA_DIR    = os.path.join(BASE_DIR, "data")      # JSON: машиночитаемые данные
  REPORTS_DIR = os.path.join(BASE_DIR, "reports")   # Markdown: документы для людей
```

Пути относительные — MCP-серверы всегда запускаются из корня проекта через Claude Code.

`_ensure_dirs()` и `save_artifact()`

```
def _ensure_dirs():
      """Создаёт все нужные папки если их нет."""
      os.makedirs(DATA_DIR, exist_ok=True)
      os.makedirs(REPORTS_DIR, exist_ok=True)



  def save_artifact(content: str, prefix: str) -> str:
      """Сохраняет Markdown-артефакт в reports/ и возвращает путь."""
      _ensure_dirs()
      timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
      filename = f"{prefix}_{timestamp}.md"
      filepath = os.path.join(REPORTS_DIR, filename)
      with open(filepath, "w", encoding="utf-8") as f:
          f.write(content)
      return f"\n\n✅ Артефакт сохранен: `{filepath}`"
```

Несколько важных следствий:

- `governance_plans/data/` и `governance_plans/reports/` **создаются автоматически** при первом вызове `save_artifact`. Ручной `mkdir` не нужен — папки уже есть в репозитории с `.gitkeep`, а при их случайном отсутствии `_ensure_dirs()` создаст их сама.
- `save_artifact` пишет **только в `reports/`** (Markdown). JSON-файлы каждый сервер пишет напрямую в `DATA_DIR` через собственную логику.
- Временна́я метка в имени файла обеспечивает уникальность: `{prefix}_20260402_143022.md`

Матрицы — единственный источник истины

Три словаря описывают ключевую бизнес-логику платформы и импортируются всеми серверами, которым они нужны:

`APPROACH_MATRIX` — выбор методологии (Predictive / Hybrid / Agile) по уровню изменений и неопределённости:

```
APPROACH_MATRIX: dict[tuple[str, str], tuple[str, list[str]]] = {
      ("Low",    "Low"):    ("Predictive (Waterfall)", [...]),
      ("Low",    "Medium"): ("Predictive (Waterfall)", [...]),
      ("Low",    "High"):   ("Hybrid",                 [...]),
      ("Medium", "Low"):    ("Hybrid",                 [...]),
      ("Medium", "Medium"): ("Hybrid",                 [...]),
      ("Medium", "High"):   ("Adaptive (Agile)",        [...]),
      ("High",   "Low"):    ("Adaptive (Agile)",        [...]),
      ("High",   "Medium"): ("Adaptive (Agile)",        [...]),
      ("High",   "High"):   ("Adaptive (Agile)",        [...]),
  }
```

`REGULATORY_OVERRIDE` — корректировка методологии при регуляторных требованиях:

```
REGULATORY_OVERRIDE: dict[str, str] = {
      "Adaptive (Agile)": "Hybrid (Agile + compliance gates)",
      "Hybrid":           "Hybrid (с усиленным Governance)",
  }
```

`QUADRANT_STRATEGIES` — стратегии вовлечения стейкхолдеров по матрице «влияние × интерес»:

```
QUADRANT_STRATEGIES: dict[tuple[str, str], tuple[str, str, str]] = {
      ("High", "High"):     ("Key Players",     "Manage Closely — ...", "Еженедельно"),
      ("High", "Medium"):   ("Context Setters", "Keep Satisfied — ...", "При вехах"),
      # ...
  }
```

> ***Правило:*** *при необходимости изменить логику выбора методологии или стратегии вовлечения — менять только здесь, в* `common.py`*. Не в каждом* `*_mcp.py` *отдельно.*

Модель `Stakeholder`

```
class Stakeholder(BaseModel):
      name: str
      influence: str = Field(..., pattern="^(Low|Medium|High)$")
      interest:  str = Field(..., pattern="^(Low|Medium|High)$")
      attitude:  Optional[str] = Field("Neutral")
```

Используется в `planning_mcp.py` для типизации входных данных реестра стейкхолдеров.

4. Архитектура MCP-серверов

Паттерн сервера

Каждый `*_mcp.py` — самодостаточный MCP-сервер. Минимальная структура:

```
# Copyright (c) 2026 Anatoly Chaussky. AI-powered Platform AInalyst. Licensed under AGP
  L v3.
  from fastmcp import FastMCP
  from skills.common import save_artifact, DATA_DIR, REPORTS_DIR

  mcp = FastMCP("babok-ch4-41")   # имя сервера — как в phase.py

  @mcp.tool()
  def save_elicitation_plan(
      project_id: str,
      stakeholder: str,
      # ...
  ) -> str:
      """Сохранить план выявления."""
      # бизнес-логика
      content = _build_markdown(...)
      return save_artifact(content, f"elicitation_plan_{project_id}_{stakeholder}")

  if __name__ == "__main__":
      mcp.run()
```

Ключевые принципы:

- **Один файл — один сервер** с собственным экземпляром `FastMCP`
- **Запуск через `mcp.run()`** в `if __name__ == "__main__"` — стандартный паттерн для Claude Code
- **Все матрицы и константы — из `common.py`**, не дублировать в каждом файле
- **Copyright-строка** в первой строке каждого файла — не трогать при правках

Почему `planning_mcp.py` — монолит

Глава 3 не разбита на отдельные серверы по задачам 3.1–3.5, в отличие от Глав 4–7. Причина: `planning_mcp.py` входит в `BASE_SERVER` и загружается во **всех** фазах. Разбивка на 5 отдельных серверов не дала бы экономии контекста — все они загружались бы в каждой фазе. При этом 6 инструментов — лёгкий монолит, архитектурно оправданный (ADR-090).

Все 22 MCP-сервера

Сервер (ключ в `.mcp.json`)	Файл	Гл. BABOK	Инструментов
`babok-ch3`	`planning_mcp.py`	3	6
`babok-confluence`	`integrations/confluence_mcp.py`	—	4
`babok-ch4-41`	`elicitation_mcp.py`	4.1	3
`babok-ch4-42`	`elicitation_conduct_mcp.py`	4.2	4
`babok-ch4-43`	`elicitation_confirm_mcp.py`	4.3	2
`babok-ch4-44`	`elicitation_communicate_mcp.py`	4.4	3
`babok-ch4-45`	`elicitation_collaborate_mcp.py`	4.5	3
`babok-ch5-51`	`requirements_traceability_mcp.py`	5.1	5
`babok-ch5-52`	`requirements_maintain_mcp.py`	5.2	4
`babok-ch5-53`	`requirements_prioritize_mcp.py`	5.3	5
`babok-ch5-54`	`requirements_assess_changes_mcp.py`	5.4	4
`babok-ch5-55`	`requirements_approve_mcp.py`	5.5	5
`babok-ch6-61`	`current_state_mcp.py`	6.1	6
`babok-ch6-62`	`future_state_mcp.py`	6.2	8
`babok-ch6-63`	`risk_assessment_mcp.py`	6.3	7
`babok-ch6-64`	`change_strategy_mcp.py`	6.4	7
`babok-ch7-71`	`requirements_spec_mcp.py`	7.1	9
`babok-ch7-72`	`requirements_verify_mcp.py`	7.2	6
`babok-ch7-73`	`requirements_validate_mcp.py`	7.3	7
`babok-ch7-74`	`requirements_architecture_mcp.py`	7.4	4
`babok-ch7-75`	`design_options_mcp.py`	7.5	5
`babok-ch7-76`	`value_recommend_mcp.py`	7.6	4

Итого: 22 сервера, 111 инструментов.

Технические ограничения FastMCP

1. `Field(..., pattern=...)` не поддерживается.

FastMCP не проходит валидацию pydantic-полей с `pattern`. Используй `Literal`:

```
# ❌ Не работает в FastMCP
  status: str = Field(..., pattern="^(Draft|Active|Approved)$")

  # ✅ Работает
  from typing import Literal
  status: Literal["Draft", "Active", "Approved"]
```

2. Сложные вложенные структуры — через JSON-строку.

Если инструмент принимает список объектов или вложенный словарь, передавай их как JSON-строку с парсингом внутри функции:

```
@mcp.tool()
  def register_requirements(project_id: str, requirements_json: str) -> str:
      """
      requirements_json: JSON-строка вида
      [{"id": "REQ-001", "text": "...", "type": "functional"}]
      """
      requirements = json.loads(requirements_json)
      # дальше работаем с Python-объектом
```

Центральный граф требований

Файл `{project_id}_traceability_repo.json` в `governance_plans/data/` — ключевой артефакт Главы 5. Формат edge list:

```
{
    "requirements": [
      {"id": "REQ-001", "text": "...", "status": "Active", "owner": "BA"}
    ],
    "links": [
      {"source": "REQ-001", "target": "REQ-002", "relation": "derives"}
    ]
  }
```

Типы связей (`relation`): `derives`, `depends`, `satisfies`, `verifies`, `modifies`.

> ***Критично:*** *ключи именно* `requirements` *(узлы) и* `links` *(рёбра), поле типа связи —* `relation`*. Не* `nodes` */* `edges`*, не* `type`*. Это формат который ожидают все инструменты Главы 5 при BFS-обходе и impact analysis.*

5. Структура скиллов (SKILL.md)

Зачем нужны скиллы

MCP-серверы выполняют операции, но не знают **методологию**: когда вызывать инструмент, в каком порядке, что спросить у BA перед вызовом. Эту роль выполняют скиллы — контекстные инструкции для Claude Code.

Скилл — это не код. Это Markdown-файл, который Claude Code читает перед началом задачи BABOK и следует его алгоритму.

Принцип progressive disclosure

Каждый скилл реализует два уровня детализации:

```
skills/
  └── elicitation_conduct/
      ├── SKILL.md            ← лёгкий контекст (4-12 КБ), всегда читается
      └── references/
          ├── single_interview.md    ← детальный справочник (~13 КБ)
          ├── multi_interview.md     ← читается только по алгоритму
          └── change_request_elicitation.md
```

- `SKILL.md` — методология, алгоритм работы, шаблоны, ссылки на инструменты. Читается Claude Code в начале каждой задачи.
- `references/*.md` — детальные справочники (гайды по RCA, методы приоритизации, критерии качества и т.д.). Читаются только когда алгоритм SKILL.md явно указывает «прочитай references/X.md».

Это снижает загрузку контекстного окна: детальный справочник по методу fishbone не нужен на каждом шаге, только когда BA реально делает RCA.

Структура SKILL.md

Каждый `SKILL.md` начинается с YAML front-matter:

```
---
  name: elicitation_conduct
  description: >
    Скилл BABOK 4.2 — Проведение выявления. Триггеры: "вот транскрипт интервью",
    "проанализируй интервью", "сравни два интервью" ...
  project: "AI-powered Platform AInalyst (AI Платформа AIналитик)"
  copyright: "Copyright (c) 2026 Anatoly Chaussky. Licensed under AGPL v3."
  ---
```

Поле `description` с триггерами — это то, по чему Claude Code определяет какой скилл нужен сейчас. Писать триггеры точно и полно — важно для правильного срабатывания.

После front-matter — содержимое в свободном Markdown:

- Роль Claude в этой задаче (одна фраза)
- Пошаговый алгоритм
- Таблицы / матрицы решений (если нужны)
- MCP-инструменты: какой вызывать, когда, с какими параметрами
- Ссылки на references с условием: «если BA выбрал метод fishbone — прочитай `references/rca_guide.md`»

Все 21 скилл

Задача BABOK	Путь к SKILL.md
Глава 3 (все задачи)	`skills/planning_prep/SKILL.md`
4.1 Подготовка к выявлению	`skills/elicitation_prep/SKILL.md`
4.2 Проведение выявления	`skills/elicitation_conduct/SKILL.md`
4.3 Подтверждение результатов	`skills/elicitation_confirm/SKILL.md`
4.4 Коммуникация результатов	`skills/elicitation_communicate/SKILL.md`
4.5 Управление сотрудничеством	`skills/elicitation_collaborate/SKILL.md`
5.1 Трассировка требований	`skills/requirements_traceability/SKILL.md`
5.2 Поддержка требований	`skills/requirements_maintain/SKILL.md`
5.3 Приоритизация	`skills/requirements_prioritize/SKILL.md`
5.4 Оценка изменений (CR)	`skills/requirements_assess_changes/SKILL.md`
5.5 Утверждение требований	`skills/requirements_approve/SKILL.md`
6.1 Анализ текущего состояния	`skills/current_state/SKILL.md`
6.2 Определение будущего состояния	`skills/future_state/SKILL.md`
6.3 Оценка рисков	`skills/risk_assessment/SKILL.md`
6.4 Стратегия изменения	`skills/change_strategy/SKILL.md`
7.1 Спецификация требований	`skills/requirements_spec/SKILL.md`
7.2 Верификация требований	`skills/requirements_verify/SKILL.md`
7.3 Валидация требований	`skills/requirements_validate/SKILL.md`
7.4 Архитектура требований	`skills/requirements_architecture/SKILL.md`
7.5 Варианты дизайна	`skills/design_options/SKILL.md`
7.6 Оценка ценности и рекомендация	`skills/value_recommend/SKILL.md`

Как добавить новый скилл

1. Создать папку `skills/{task_name}/`
2. Создать `SKILL.md` с YAML front-matter (name, description с триггерами, project, copyright)
3. Опционально — создать `skills/{task_name}/references/` с детальными справочниками
4. Добавить ссылку на скилл в `CLAUDE.md` (таблица «Скиллы — читай перед каждой задачей»)

---

6. Хранилище артефактов

Структура папок

```
governance_plans/
  ├── data/                                    ← JSON, машиночитаемые данные для MCP
  |    ├── .gitkeep
  |    ├── {project}_traceability_repo.json    ← граф требований (5.1)
  |    ├── {project}_prioritization.json       ← результаты приоритизации (5.3)
  |    ├── {project}_design_options.json       ← варианты дизайна (7.5)
  |    └── {project}_change_strategy.json      ← стратегия изменения (6.4)
  └── reports/                                 ← Markdown, документы для людей
       ├── .gitkeep
       ├── {project}_ba_approach.md            ← выбор подхода (3.1)
       ├── {project}_stakeholder_plan.md       ← карта стейкхолдеров (3.2)
       ├── 6_1_current_state_{project}.md      ← анализ as-is (6.1)
       ├── 6_3_risk_assessment_{project}.md    ← оценка рисков (6.3)
       └── 7_6_recommendation_{project}.md     ← рекомендация спонсору (7.6)
```

Правило разделения: JSON → `data/`, Markdown → `reports/`. Это зафиксировано в `common.py` через константы `DATA_DIR` и `REPORTS_DIR` и отражено в `.gitignore`.

`.gitignore` и `.gitkeep`

```
governance_plans/data/*
  governance_plans/reports/*
  !governance_plans/data/.gitkeep
  !governance_plans/reports/.gitkeep
```

Содержимое папок игнорируется Git — артефакты специфичны для проекта конкретного BA и не должны попадать в репозиторий. Сами папки сохраняются через `.gitkeep`.

`_ensure_dirs()` в `common.py` создаёт `data/` и `reports/` автоматически при первом вызове `save_artifact`. Папки уже присутствуют в репозитории с `.gitkeep` — ручной `mkdir` не нужен.

`inputs/`

```
inputs/
  ├── README.md        ← инструкция для BA что сюда класть
  ├── interview_ivanov_21mar.txt
  ├── workshop_results.pdf
  └── regulations_v3.docx
```

BA кладёт сюда входные материалы перед обработкой. Содержимое в `.gitignore`. Claude Code читает файлы напрямую по пути — BA достаточно назвать имя файла в разговоре.

Поддерживаемые форматы: `.txt`, `.md`, `.pdf`, `.docx`

Экспорт PDF

```
python export_pdf.py stakeholder_plan.md           # один файл
  python export_pdf.py --all                         # все .md из reports/ (спросит пер
  ед перезаписью)
  python export_pdf.py --all --force                 # без подтверждения
```

PDF создаётся рядом с `.md`-файлом в `governance_plans/reports/`. В Git не попадает.

Зависимость: `reportlab`. Устанавливается вместе с `requirements.txt`.

Три уровня хранения

Уровень	Где	Статус
1 — рабочее	`governance_plans/` на машине BA	✅ Реализовано
2 — командное	Confluence (через `confluence_mcp.py`)	✅ Реализовано
3 — версионное	Git (история изменений, аудит)	📋 Технический долг

---

7. Интеграция с Confluence

Архитектура

`skills/integrations/confluence_mcp.py` — входит в `BASE_SERVER` и **загружается во всех фазах автоматически**. Это не опциональный плагин, а часть базовой конфигурации.
Сервер стартует без ошибок даже при незаполненном `.env` — graceful fallback. Ошибка подключения возникает только в момент вызова конкретного инструмента, когда сервер пытается обратиться к Confluence API.

4 инструмента

Инструмент	Что делает
`push_to_confluence`	Публикует Markdown-артефакт как страницу Confluence
`pull_from_confluence`	Загружает содержимое страницы в контекст
`sync_page`	Обновляет существующую страницу (или создаёт если нет)
`list_space_pages`	Список страниц в пространстве (поиск по заголовку)

Настройка через `.env`

Скопировать `.env.example` → `.env` и заполнить один из двух вариантов:

Confluence Cloud:

```
CONFLUENCE_URL=https://your-company.atlassian.net
  CONFLUENCE_EMAIL=you@company.com
  CONFLUENCE_API_TOKEN=your_api_token_here
  CONFLUENCE_SPACE_KEY=BA
```

Confluence Server / Data Center:

```
CONFLUENCE_URL=https://confluence.internal.company.com
  CONFLUENCE_USERNAME=your_username
  CONFLUENCE_PASSWORD=your_password
  CONFLUENCE_SPACE_KEY=BA
```

`CONFLUENCE_SPACE_KEY` — пространство по умолчанию. Можно переопределить при каждом вызове инструмента через параметр `space_key`.

Типичный сценарий использования

После создания артефакта в `governance_plans/reports/` BA может опубликовать его в Confluence одной командой Claude Code:

> *«Опубликуй план стейкхолдеров в Confluence, пространство BA»*

Claude Code вызывает `push_to_confluence` — Markdown конвертируется в Confluence Storage Format через `_markdown_to_confluence_storage()` и публикуется как страница.

---

8. Тестирование

Запуск

```
python3 -m unittest discover
```

Без pip, без внешних зависимостей — все внешние пакеты замоканы в `conftest.py`. Тесты запускаются в чистом Python-окружении.

`conftest.py` и моки

`conftest.py` мокает все внешние зависимости до импорта MCP-серверов:

```
# Мокаются: fastmcp, pydantic, mcp, atlassian, markdown2
  sys.modules["fastmcp"] = MagicMock()
  sys.modules["pydantic"] = MagicMock()
  # ...
```

Это позволяет запускать тесты без установки `mcp[cli]`, `fastmcp`, `atlassian-python-api` и других runtime-зависимостей.

`BaseMCPTest`

Базовый класс для всех тестов платформы:

```
class BaseMCPTest(unittest.TestCase):
      def setUp(self):
          self.test_dir = tempfile.mkdtemp()
          self.original_dir = os.getcwd()
          os.chdir(self.test_dir)          # каждый тест изолирован в tmpdir

      def tearDown(self):
          os.chdir(self.original_dir)
          shutil.rmtree(self.test_dir)
```

Каждый тест работает в своём временном каталоге — артефакты не засоряют рабочую папку и не влияют друг на друга.

ADR-068: `patch` вместо глобального мока

```
# ✅ Правильно — патчим save_artifact в конкретном модуле
@patch("skills.elicitation_mcp.save_artifact")
def test_save_plan(self, mock_save):
    mock_save.return_value = "✅ Артефакт сохранен"
    result = save_elicitation_plan(project_id="test", ...)
    mock_save.assert_called_once()

# ❌ Неправильно — глобальный мок ломает другие тесты
@patch("skills.common.save_artifact")
```

Патчить нужно в пространстве имён того модуля, который использует функцию, а не там где она определена.

Структура тестов

Каждая глава и задача BABOK имеет отдельный файл:

Файл	Покрывает
`tests/test_ch3_ch4.py`	Глава 4 (Глава 3 тестируется через `planning_mcp.py`)
`tests/test_ch4_41.py` ... `test_ch4_45.py`	Задачи 4.1–4.5 отдельно
`tests/test_ch5_51.py` ... `test_ch5_55.py`	Задачи 5.1–5.5 отдельно
`tests/test_ch6.py`	Глава 6 общий
`tests/test_ch7.py`	Глава 7 общий
`tests/test_confluence.py`	Интеграция Confluence

Текущее покрытие (v27): 1636 тестов, все зелёные.

Как добавить тест для нового инструмента

```
from tests.conftest import BaseMCPTest
  from unittest.mock import patch
  from skills.my_new_mcp import my_new_tool   # импорт после моков в conftest

  class TestMyNewTool(BaseMCPTest):

      def setUp(self):
          super().setUp()
          # дополнительная настройка если нужна

      @patch("skills.my_new_mcp.save_artifact")
      def test_basic_call(self, mock_save):
          mock_save.return_value = "✅ Артефакт сохранен"
          result = my_new_tool(project_id="test_proj", param="value")
          self.assertIn("test_proj", result)
          mock_save.assert_called_once()
```

---

9. Среда разработки

Требования

- Python 3.10 или выше
- pip (входит в стандартную поставку Python)

Установка зависимостей

```
pip install -r requirements.txt
```

`requirements.txt` содержит зафиксированные версии всех зависимостей (ADR из REVIEW_v20):

```
mcp[cli]==1.6.0
  fastmcp==2.3.3
  pydantic==2.11.1
  atlassian-python-api==3.41.16  # Confluence Cloud + Server/DC интеграция
  markdown2==2.5.3               # Markdown → HTML для Confluence storage format
  reportlab==4.2.5               # PDF-экспорт отчётов
```

Версии зафиксированы намеренно — плавающие ограничения (`>=`) допускают breaking-изменения при обновлении. При изменении версий — проверять совместимость вручную.

Первый запуск

```
git clone <url>
  cd ainalyst
  pip install -r requirements.txt
  cp .env.example .env
  # Заполнить .env — Confluence API-ключи (если нужна интеграция)
  python phase.py planning
  # Открыть проект в Claude Code
```

После этих шагов `.mcp.json` сгенерирован с правильными абсолютными путями для текущей машины. Claude Code при открытии проекта загрузит MCP-серверы фазы `planning`.

Почему `.mcp.json` не в репозитории

`.mcp.json` содержит абсолютные пути, специфичные для машины разработчика. Захардкоженный путь `/home/claude/ainalyst/` сломает Claude Code у любого, кто клонирует репозиторий в другую директорию (DAMAGE*REPORT*v26, Проблема 2). Файл добавлен в `.gitignore` — он генерируется `phase.py` при первом запуске.
Аналогично в `.gitignore` попадает `.ainalyst_phase` — файл хранит имя текущей активной фазы и специфичен для сессии разработчика.

Настройка `.env`

Скопировать `.env.example` → `.env`. Два варианта в зависимости от типа Confluence:

Confluence Cloud:

```
CONFLUENCE_URL=https://your-company.atlassian.net
  CONFLUENCE_EMAIL=you@company.com
  CONFLUENCE_API_TOKEN=your_api_token_here
  CONFLUENCE_SPACE_KEY=BA
```

Confluence Server / Data Center:

```
CONFLUENCE_URL=https://confluence.internal.company.com
  CONFLUENCE_USERNAME=your_username
  CONFLUENCE_PASSWORD=your_password
  CONFLUENCE_SPACE_KEY=BA
```

Если `.env` не заполнен — платформа работает в полном объёме кроме Confluence-инструментов. MCP-сервер `confluence_mcp.py` стартует без ошибок, ошибка подключения возникает только в момент вызова конкретного инструмента.

`.claude/` — интеграция с Claude Code

```
.claude/
  ├── settings.json        ← разрешения и настройки Claude Code
  ├── hooks/
  │   ├── session_start.sh ← запускается при старте сессии
  │   └── post_tool_use.sh ← запускается после каждого вызова инструмента
  └── rules/
      ├── artifacts.md     ← правила работы с артефактами
      └── babok_process.md ← правила соблюдения BABOK-процесса
```

`settings.json` объявляет хуки и привязывает их к событиям Claude Code (`SessionStart`, `PostToolUse`). При добавлении нового хука — регистрировать здесь.

`session_start.sh` — запускается в начале каждой сессии. Выводит в контекст Claude Code:

- список активных проектов (по JSON-файлам в `governance_plans/data/`)
- последние 5 артефактов из `governance_plans/reports/`
- список входных материалов в `inputs/`, готовых к обработке
- подсказки по командам (голосовой режим, плановый режим, экспорт PDF)

Использует `find` вместо `ls *.{ext}` — исправленная реализация (ADR из REVIEW_v20, п.7). Раньше bash brace expansion давал ошибку если файлов одного из типов нет.

`post_tool_use.sh` — запускается после каждого вызова MCP-инструмента. Если инструмент сохранил `.md`-файл в `governance_plans/reports/` — выводит уведомление с именем файла и командой для просмотра.

`rules/` — правила поведения Claude Code, которые он учитывает при работе. `artifacts.md` описывает как именовать и сохранять артефакты; `babok_process.md` — правила следования BABOK-методологии в диалоге с BA.

Смена фазы в процессе разработки

```
python phase.py design     # 1. Генерирует новый .mcp.json
  # В Claude Code: /restart  # 2. Перезагружает MCP-серверы
```

Смена фазы без `/restart` не применяется — Claude Code держит серверы загруженными до явной перезагрузки.

---

10. Добавление нового MCP-сервера

Чеклист (6 шагов)

Шаг 1 — Создать MCP-сервер

Файл `skills/{chapter}_{task}_mcp.py`. Минимальный шаблон:

```
# Copyright (c) 2026 Anatoly Chaussky. All rights reserved.
  # Licensed under the AInalyst Commercial License (see COMMERCIAL_LICENSE.md).
  """
  AInalyst — BABOK Глава X.Y: [Название задачи]
  MCP-сервер для Claude Code.
  """

  from fastmcp import FastMCP
  from skills.common import save_artifact, DATA_DIR, REPORTS_DIR

  mcp = FastMCP("babok-chX-XY")



  @mcp.tool()
  def my_new_tool(
      project_id: str,
      param: str,
  ) -> str:
      """
      [Описание инструмента для Claude Code].

      Возвращает путь к сохранённому артефакту.
      """
      content = f"# Результат\n\nproject_id: {project_id}\n"
      return save_artifact(content, f"{project_id}_my_prefix")



  if __name__ == "__main__":
      mcp.run()
```

Требования:

- **Copyright-строка** в начале файла — не трогать при правках
- `FastMCP` с уникальным именем сервера (`babok-chX-XY`)
- `save_artifact` из `common.py` — не писать файлы напрямую

- Все матрицы и константы — в `common.py`, не дублировать в сервере

Шаг 2 — Создать скилл

```
skills/{task}/
  ├── SKILL.md          ← всегда. Методология, алгоритм, ссылки на MCP
  └── references/       ← опционально. Детальные справочники
      ├── guide.md
      └── templates.md
```

Структура `SKILL.md` (минимум):

```
# [Название задачи BABOK]

  ## Что это
  [Одна фраза — суть задачи]

  ## Когда применять
  [Триггеры — когда BA запрашивает эту задачу]

  ## Алгоритм
  1. [Шаг 1]
  2. [Шаг 2]
  3. Вызвать `my_new_tool(project_id=..., param=...)`

  ## MCP-инструменты
  | Инструмент | Когда |
  |---|---|
  | `my_new_tool` | [условие] |

  ## Справочники
  - `references/guide.md` — читать когда [условие]
```

Шаг 3 — Зарегистрировать в `phase.py`

Добавить сервер в нужную фазу:

```
"analysis": {
      "servers": {
          **BASE_SERVER,
          "babok-ch6-61": _server("skills/current_state_mcp.py"),
          "babok-chX-XY": _server("skills/my_new_mcp.py"),   # ← добавить здесь
      }
  },
```

Если сервер нужен во всех фазах — добавить в `BASE_SERVER`. Но это редкий случай: в `BASE_SERVER` сейчас только 2 сервера (`planning_mcp.py` и `confluence_mcp.py`).

Шаг 4 — Обновить `CLAUDE.md`

Добавить инструмент в таблицу серверов и прописать триггеры — когда Claude Code должен вызывать этот инструмент. `CLAUDE.md` — системный промпт агента, именно он определяет поведение Claude Code при запросах BA.

Шаг 5 — Написать тесты

Файл `tests/test_chX_XY.py`:

```
# Copyright (c) 2026 Anatoly Chaussky. All rights reserved.
  import sys
  import os
  sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

  from tests.conftest import BaseMCPTest
  from unittest.mock import patch
  import skills.my_new_mcp as mod


  class TestMyNewTool(BaseMCPTest):

      def _call(self, **overrides):
          # ADR-084: {**defaults, **overrides} — не dict(key=val, **overrides)
          defaults = {
              "project_id": "test_proj",
              "param": "value",
          }
          kwargs = {**defaults, **overrides}
          with patch("skills.my_new_mcp.save_artifact") as mock_save:
              mock_save.return_value = "✅ Артефакт сохранен"
              result = mod.my_new_tool(**kwargs)
              return result, mock_save

      def test_basic_call(self):
          result, mock_save = self._call()
          self.assertIn("test_proj", result)
          mock_save.assert_called_once()

      def test_project_id_in_result(self):
          result, _ = self._call(project_id="crm_bank")
          self.assertIn("crm_bank", result)
```

Паттерн `_call(**overrides)` — обязательный (ADR-084). `{**defaults, **overrides}` семантически корректен и не падает при передаче уже существующего ключа.

Шаг 6 — Запустить тесты

```
python3 -m unittest discover
```

Все 1636+ тестов должны быть зелёными после добавления нового сервера.

Требования к именованию

Сущность	Паттерн	Пример
MCP-файл	`skills/{chapter}_{task}_mcp.py`	`skills/risk_assessment_mcp.py`
FastMCP имя	`babok-chX-XY`	`babok-ch6-63`
Скилл-папка	`skills/{task}/`	`skills/risk_assessment/`
Тест-файл	`tests/test_chX_XY.py`	`tests/test_ch6_63.py`
JSON-артефакт	`{project_id}_{prefix}.json`	`crm_bank_risk_register.json`
MD-артефакт	`{X_Y}_{description}_{project}.md`	`6_3_risk_assessment_crm_bank.md`

---

11. Технический долг и ADR-реестр

Ключевые ADR

Полный реестр — в `DECISIONS.md`. Ниже — решения, наиболее важные для понимания архитектуры.

ADR-090 — Удаление `main.py` (Сессия 45, 02.04.2026)

`main.py` был легаси-обёрткой, реэкспортировавшей функции из `planning_mcp.py`. Удалён: создавал путаницу по точке входа, «обратная совместимость» была бессмысленна (нет публичного API). Глава 3 обслуживается исключительно `skills/planning_mcp.py`.

ADR-089 — Удаление `planning.py` (Сессия 46, 02.04.2026, REVIEW_v26)

`planning.py` был «чистым» утилитным модулем бизнес-логики Главы 3 без MCP-обёртки. Использовался только в `tests/test_ch3_ch4.py`. Дублировал `_classify_stakeholder` из `planning_mcp.py` с другой сигнатурой (принимала объект `Stakeholder` вместо двух строк). Удалён: `tests/test_ch3_ch4.py` переписан под прямое тестирование `planning_mcp.py` через `BaseMCPTest`. Архитектура Главы 3 приведена в соответствие с остальными главами.

ADR-088 — `planning_mcp.py` остаётся монолитом (Сессия 46)

Принято решение не разбивать `planning_mcp.py` на 5 серверов по задачам 3.1–3.5, несмотря на то что Главы 4–7 устроены именно так. Обоснование: `planning_mcp.py` входит в `BASE_SERVER` и присутствует во всех фазах — разбивка не даёт экономии контекстного окна. 6 инструментов — лёгкий сервер, монолит здесь оправдан. Симметрия ради симметрии избыточна.

ADR-085 — Расхождения сигнатур при написании тестов Главы 5 (Сессия 38, 29.03.2026)

При написании `test_ch5_51.py`–`test_ch5_53.py` обнаружены расхождения между ожидаемыми и фактическими сигнатурами инструментов. Зафиксировано как паттерн: тесты пишутся по реальным сигнатурам кода, не по документации. При рефакторинге инструментов — обновлять тесты одновременно с кодом.

ADR-084 — Паттерн `_call(**overrides)` в тест-классах (Сессия 39)

`dict(key=val, **overrides)` падает с `TypeError` при передаче ключа уже присутствующего в `dict()`. Обязательный паттерн для всех `_call(**overrides)`: `{**defaults, **overrides}`. `overrides` побеждает — конфликтов нет. Зафиксировано в `conftest.py` как комментарий.

ADR-068 — `patch` вместо глобального мока (ранние сессии)

При тестировании MCP-серверов патчить нужно в пространстве имён того модуля, который использует функцию: `@patch("skills.my_mcp.save_artifact")`, а не `@patch("skills.common.save_artifact")`. Глобальный мок `common.save_artifact` ломает другие тесты, запущенные параллельно или в одном discover-проходе.

ADR-047–053 — Claude-in-Claude (статус: 📋 Проектирование)

Серия решений по архитектуре функции Claude-in-Claude — вызов вложенного Claude-агента из MCP-инструмента для сложных аналитических операций. Единственный незакрытый функциональный блок платформы. Оставлен на последнюю очередь разработки.

Открытый технический долг

#	Проблема	Критичность	Статус
1	Claude-in-Claude (ADR-047–053)	🔴 Функционал	Последняя очередь
2	Уровень 3 хранилища (Git-версионирование артефактов)	🔵 Архитектура	Не реализован
3	Запустить `pytest` на реальной машине после публикации на GitHub	📋 Процесс	После GitHub
4	Стратегия обновления платформы без потери данных проекта	🔵 Архитектура	Требует проектирования
5	`_classify_stakeholder` с двумя сигнатурами удалён, но общая проверка сигнатур после рефакторинга не проводилась	📋 QA	После публикации

Уровень 3 хранилища (Git-версионирование) — артефакты `governance_plans/` игнорируются Git по умолчанию (`.gitignore`). Планировалась возможность вести историю изменений и аудит через Git. Варианты реализации: отдельная ветка под данные проекта, отдельный репозиторий, `git add -f` для явного включения артефактов. Решение не принято, реализация отложена.
Стратегия обновления платформы — BA работает над проектом в копии `v23`, выходит `v24`. Как получить новые возможности не потеряв артефакты в `governance_plans/` и входные материалы в `inputs/`? Рассматриваемые варианты: `git pull` (требует git-грамотности BA), скрипт `update.py`, физическое разделение платформы и данных. Требует отдельного ADR. Подробности — в `DECISIONS.md`, раздел IDEA «Workflow: новый проект и обновление платформы».

Changelog по версиям

v20 → v26 (март 2026)

Основные изменения:

- Написаны отдельные тесты для Глав 4 и 5 (было: монолитный `test_ch3_ch4.py`). Добавлено ~156 тестов для задач 5.1–5.3
- Добавлен `README.md` в корне проекта
- Исправлен glob-баг в `session_start.sh` (`find` вместо `ls *.{txt,md,pdf,docx}`)
- Зафиксированы точные версии в `requirements.txt` (были плавающие `>=`)
- Добавлен `export_pdf.py` — конвертер `governance_plans/reports/*.md` → PDF
- Переименована `interviews/` → `inputs/` (расширен список типов входных данных)
- Добавлена пользовательская инструкция (`USER_GUIDE.md`) и 6 отдельных CH-файлов
- Удалён `.mcp.json` из репозитория, добавлен в `.gitignore` — генерируется `phase.py`
- Созданы `governance_plans/data/.gitkeep` и `governance_plans/reports/.gitkeep`

v26 → v27 (1–2 апреля 2026, финальная подготовка к GitHub)

- Исправлены все критические баги из `DAMAGE_REPORT_v26.md`
- Удалён `main.py` (ADR-090) — Глава 3 через `planning_mcp.py`
- Удалён `planning.py` (ADR-089) — дублирование с `planning_mcp.py` устранено
- `tests/test_ch3_ch4.py` переписан под `BaseMCPTest` + прямое тестирование `planning_mcp.py`
- `CLAUDE.md` исправлен: структура `governance_plans/` теперь отражает реальные подпапки `data/` и `reports/`
- `phase.py` проверен: все 22 сервера на диске, пути через `Path(__file__).resolve().parent`
- Итог: 1636 тестов, все зелёные; 22 MCP-сервера, 111 инструментов; 27 Python-файлов без синтаксических ошибок; захардкоженные пути `/home/claude` не обнаружены